

DEVOPS - TOOLS

Instructor Lucas Bonanni

TOOLS OVERVIEW

TOOLS OVERVIEW

- **Git:** Sistema de control de versiones
- **Github:** Plataforma de hosting y colaboración
- **Docker:** Plataforma de virtualización de contenedores

GIT

INTRODUCTION

Git es un sistema de control de versiones ampliamente utilizado en el desarrollo de software y otras áreas de tecnología.

- Permite rastrear los cambios realizados en el código fuente a lo largo del tiempo
- Fue creado por Linus Torvalds en 2005 para gestionar el desarrollo del kernel de Linux

VERSION CONTROL AND COLLABORATION

Git facilita la gestión de cambios en el código fuente:

- Permite a los desarrolladores trabajar de manera colaborativa en un proyecto
- Proporciona un historial completo de todos los cambios realizados
- Facilita la identificación y reversión de errores

VERSION CONTROL SYSTEM (VCS)

Un VCS gestiona la evolución de archivos en el tiempo y conserva historial completo de cambios.

Objetivos principales:

- Trazabilidad de quién cambió qué y cuándo
- Colaboración segura entre múltiples personas
- Recuperación de versiones estables ante errores
- Soporte a branching, merging y release management

Beneficios:

- Menor riesgo de pérdida de trabajo
- Auditoría técnica del proyecto
- Más velocidad para experimentar y corregir

VCS TYPES AND ARCHITECTURE

Tipos clásicos de sistemas de control de versiones:

- **Local VCS:** historial en una sola máquina
- **Centralized VCS (CVCS):** un servidor central con repositorio único
- **Distributed VCS (DVCS):** cada desarrollador tiene copia completa del repositorio

Centralized vs Distributed:

- CVCS: depende más del servidor central para operar
- DVCS: permite trabajo offline y mayor resiliencia
- CVCS: colaboración centralizada; DVCS: colaboración flexible entre múltiples remotos

MAIN FEATURES

- **Sistema distribuido:** Cada desarrollador tiene una copia completa del repositorio
- **Rendimiento:** Rápida velocidad de rendimiento y eficiencia en la gestión de grandes proyectos
- **Funcionalidades:** Amplia gama de comandos (branches, tags, merges)
- **Flexibilidad:** Se adapta a diferentes flujos de trabajo y necesidades de desarrollo

GIT ESSENTIALS (COMANDOS CLAVE)

Comandos mínimos para el día a día:

- `git init`: Inicializa un repositorio local
- `git clone <repo-url>`: Clona un repositorio remoto
- `git add .` o `git add <archivo>`: Prepara cambios para commit
- `git commit -m "mensaje"`: Registra cambios en el repositorio local
- `git status`: Muestra el estado actual del working tree
- `git log --oneline`: Resume historial de commits
- `git pull`: Trae y fusiona cambios del remoto
- `git push`: Publica commits locales en el repositorio remoto

GIT EXAMPLE (CLI)

Ejemplo rápido de trabajo local + remoto:

```
1 git clone https://github.com/acme/mi-app.git
2 cd mi-app
3 git checkout -b feature/login
4 git add .
5 git commit -m "feat: agregar pantalla de login"
6 git push -u origin feature/login
```

CONVENTIONAL COMMITS

Convención para escribir mensajes de commit claros y automatizables.

Formato:

- `<tipo>(scope opcional): descripción`

Tipos comunes:

- `feat`: nueva funcionalidad
- `fix`: corrección de bug
- `docs`: cambios de documentación
- `chore`: tareas de mantenimiento
- `refactor`: mejora interna sin cambiar comportamiento

COMMIT MESSAGE BEST PRACTICES

Buenas prácticas para mensajes de commit:

- Usar verbo en imperativo: **add**, **fix**, **remove**, **update**
- Escribir una idea por commit (cambios pequeños y atómicos)
- Mantener el título breve y claro (idealmente ≤ 72 caracteres)
- Explicar el *qué* y, cuando haga falta, el *por qué*
- Evitar mensajes genéricos como **changes**, **fix**, **update** **stuff**

Estructura recomendada:

- Título corto
- Línea en blanco
- Cuerpo opcional con contexto técnico

CONVENTIONAL COMMITS IN PRACTICE

Patrón recomendado:

- `<type>(scope opcional): descripción`

Ejemplos:

- `feat(auth): add OAuth login`
- `fix(api): handle timeout in users endpoint`
- `docs(readme): update local setup steps`

Beneficio principal:

- Estandariza historial, facilita revisiones y mejora automatización en CI/CD.

CONVENTIONAL COMMITS - EXAMPLES

Ejemplos reales:

```
1 git commit -m "feat(auth): agregar login con OAuth"  
2 git commit -m "fix(api): corregir timeout en endpoint de usuarios"  
3 git commit -m "docs(readme): actualizar guía de instalación"  
4 git commit -m "chore(deps): actualizar dependencias de seguridad"
```

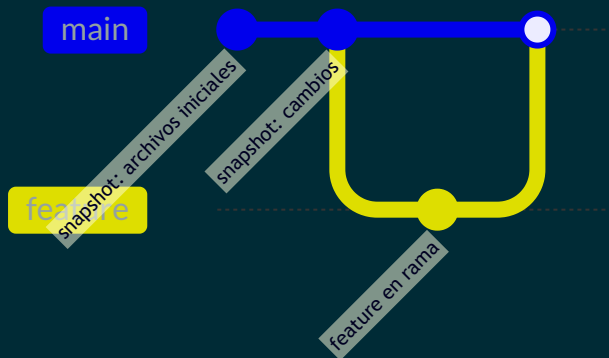
Beneficios:

- Historial más legible para el equipo
- Facilita changelogs automáticos
- Mejora trazabilidad para CI/CD

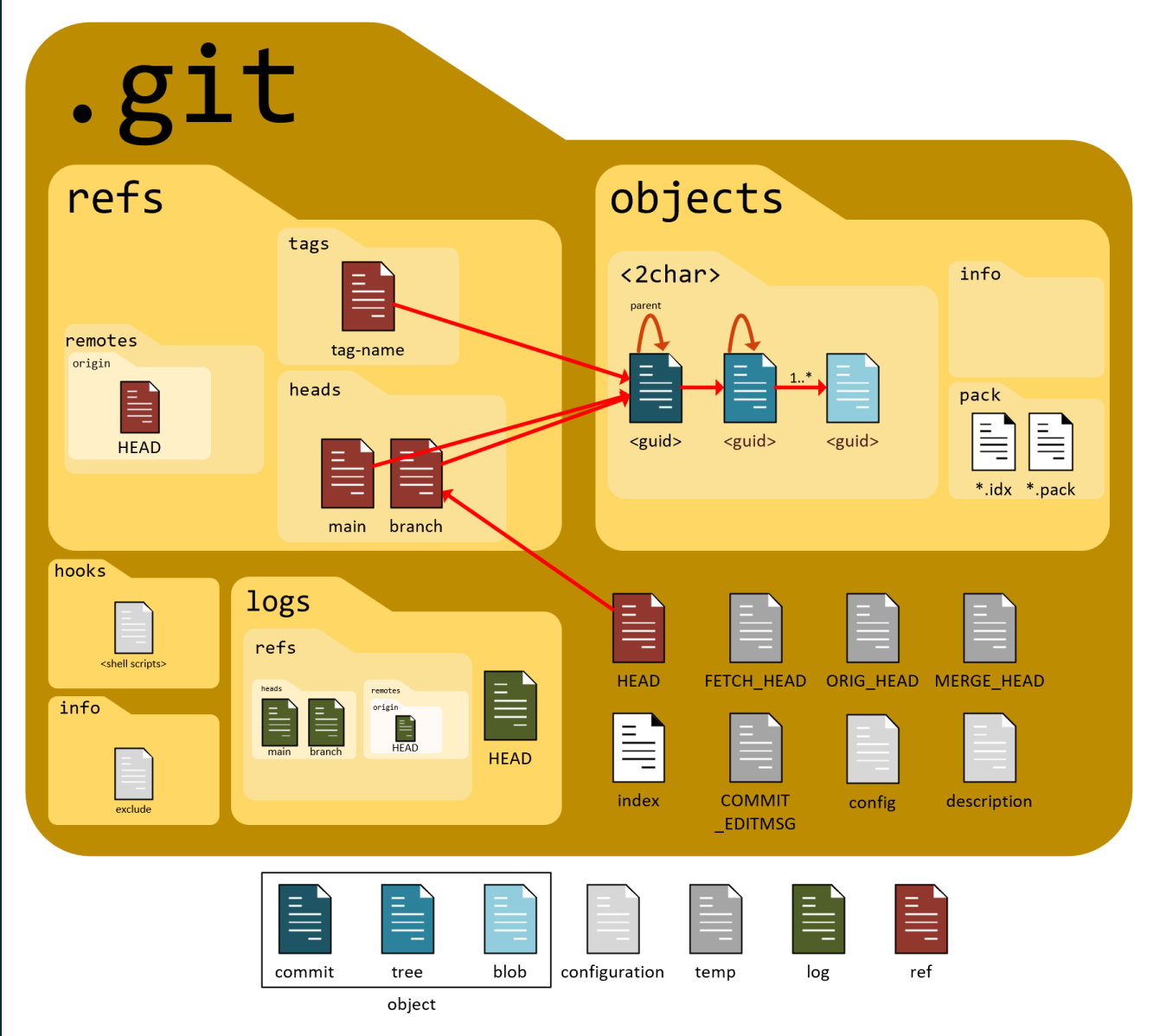
GIT DATA MODEL CONCEPT

Git modela el historial como un grafo de snapshots inmutables.

- **Blob:** contenido de archivo
- **Tree:** estructura de directorios y blobs
- **Commit:** snapshot con autor, fecha, mensaje y referencia al commit padre
- **Branch:** puntero móvil al commit más reciente de una línea de trabajo
- **HEAD:** referencia a la rama/commit actualmente checkout



GIT DATA MODEL CONCEPT (CONTINUACIÓN)



GIT TAGS AND RELEASES

Un tag es una referencia fija a un commit específico.

Uso típico:

- Marcar versiones de release (**v1.0.0**, **v1.1.0**)
- Facilitar rollback a versiones conocidas
- Asociar artefactos y notas de lanzamiento a un punto exacto del historial

Comandos comunes:

- **git tag v1.0.0**
- **git push origin v1.0.0**
- **git checkout v1.0.0**

BRANCHES

Una característica destacada es la posibilidad de crear ramas para trabajar en diferentes aspectos del proyecto de manera aislada.

- Las ramas permiten trabajar en nuevas características o correcciones sin afectar la rama principal (main)
- Las ramas se pueden fusionar posteriormente para incorporar los cambios realizados

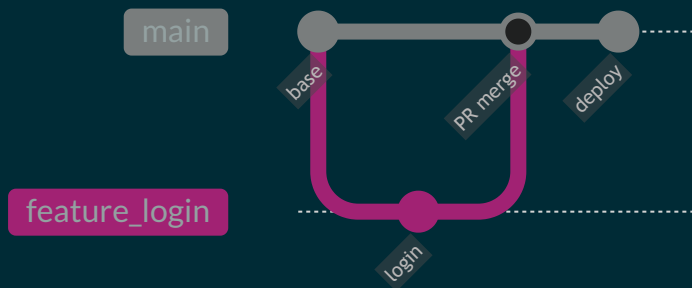
GITHUB FLOW

Descripción:

- Una rama corta por cambio y Pull Request hacia **main**
- Integración frecuente y despliegue continuo

Cuándo usar:

- Equipos pequeños/medianos con releases frecuentes



GIT FLOW

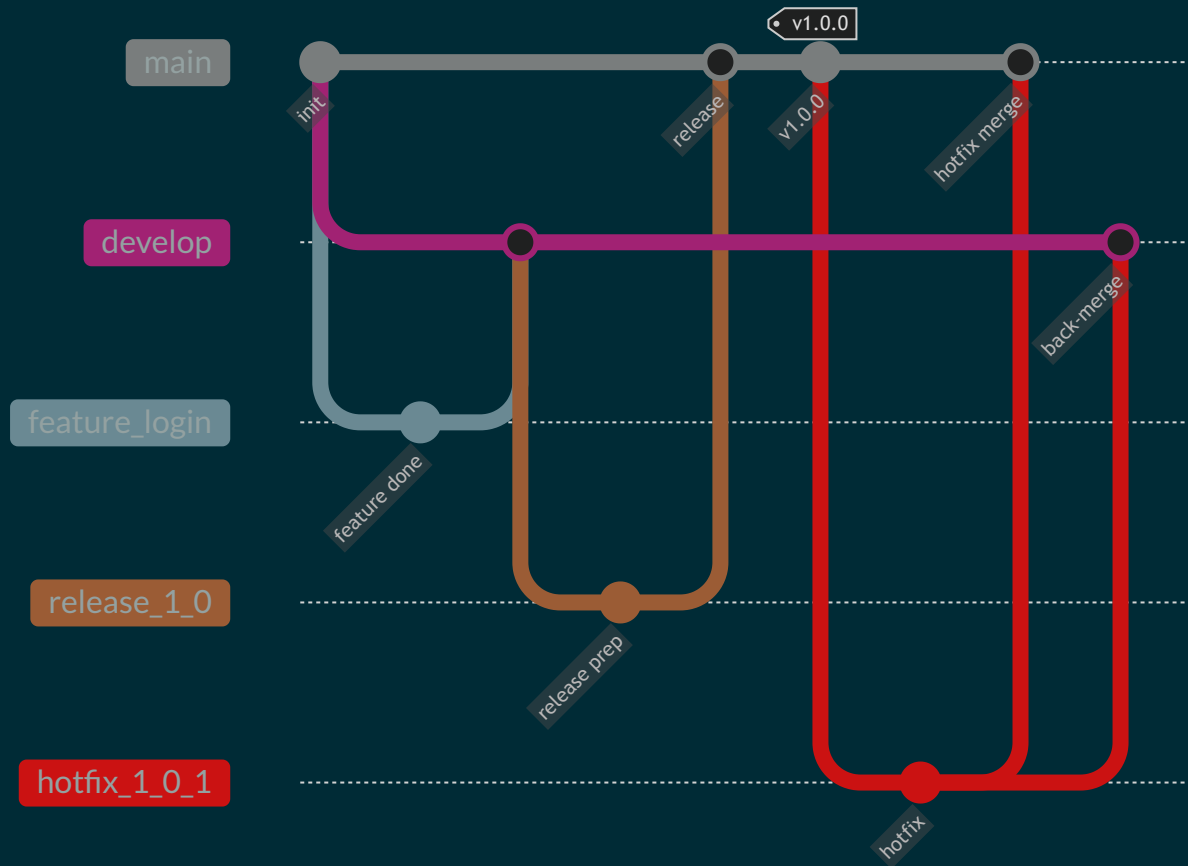
Descripción:

- Ramas dedicadas para `feature/*`, `release/*` y `hotfix/*`
- Flujo más estructurado para ciclos de release formales

Cuándo usar:

- Productos con versionado y ventanas de publicación definidas

GIT FLOW (CONTINUACIÓN)



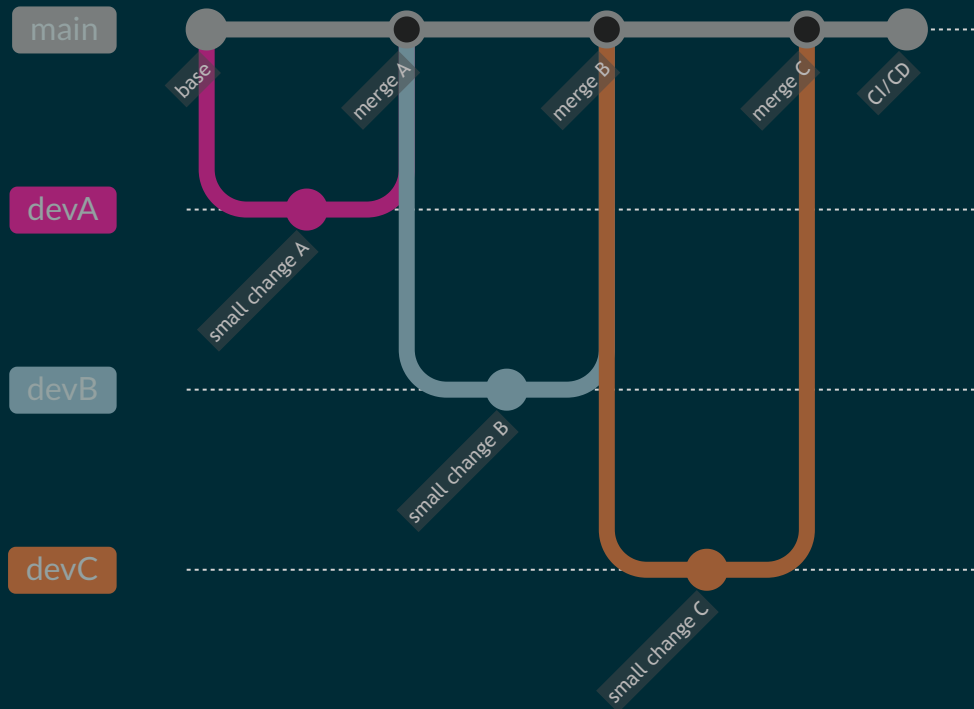
TRUNK-BASED DEVELOPMENT

Descripción:

- Cambios pequeños y frecuentes sobre **main**
- Ramas de vida muy corta o integración directa con feature flags

Cuándo usar:

- Equipos con fuerte automatización de CI/CD



POPULAR REPOSITORY HOSTING PLATFORMS

Existen diversas plataformas populares para alojar repositorios:

- **GitHub**
- **GitLab**
- **Bitbucket**

Estas plataformas ofrecen funcionalidades adicionales:

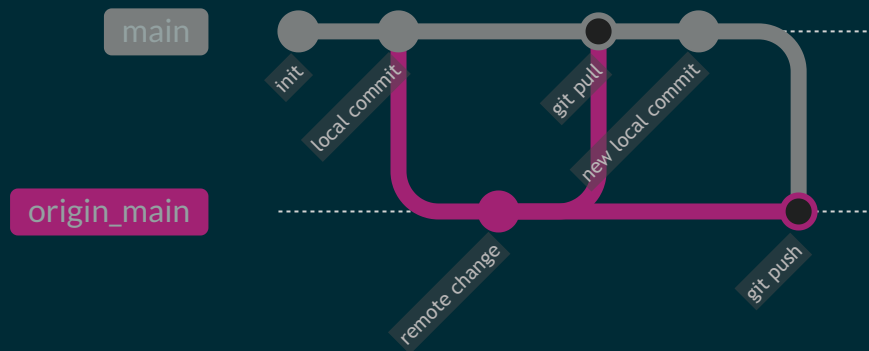
- Colaboración en equipo
- Seguimiento de problemas
- Revisión de código

GIT WORKFLOW

Flujo de trabajo típico con Git:

- Working Directory
- Staging Area
- Local Repository
- Remote Repository

GIT WORKFLOW DIAGRAM



GIT LINKS

Recursos recomendados:

- [Try Git](#) - learn Git in 15 minutes with pseudo-terminal interface
- [Atlassian Git Tutorial](#) - comprehensive tutorial on Git
- [Pro Git book](#) - free Git book, also available on paper (CC BY-NC-SA 3.0)
- [So You Think You Know Git Part 2 - DevWorld 2024](#)
- [So You Think You Know Git - FOSDEM 2024](#)
- [Git Config | Ep. 14 Bits and Booze](#)
- [How Core Git Developers Configure Git | Butler's Log](#)
- [Advanced Git Tutorial - Interactive Rebase, Cherry-Picking, Reflog, Submodules and more](#)
- [Learn Git Branching - interactive](#)
- [awesome-git](#) - Repository with resources

GITHUB

KEY GITHUB FEATURES

- **Repositorios:** Alojar proyectos y gestionar el control de versiones utilizando Git
- **Colaboración:** Trabajar en equipo, realizar revisiones de código
- **Issues:** Seguimiento de problemas, tareas y mejoras
- **Pull Requests:** Revisión y aprobación de cambios propuestos
- **Integraciones:** Amplia gama de herramientas y servicios
- **Wikis y documentación:** Creación de documentación asociada a proyectos

FIRST STEPS ON GITHUB

Flujo inicial recomendado para comenzar un proyecto:

1. Crear un repositorio en GitHub
2. Clonarlo localmente con `git clone`
3. Hacer cambios y crear commits en una rama
4. Publicar la rama con `git push`
5. Abrir una Pull Request para revisión e integración

PULL REQUEST EXAMPLE

Comandos típicos antes de abrir una Pull Request:

```
1 git checkout -b feature/monitoring-basics
2 git add .
3 git commit -m "docs: add monitoring basics slide"
4 git push -u origin feature/monitoring-basics
```

Luego, en GitHub:

1. Crear Pull Request hacia main
2. Solicitar review
3. Hacer merge al aprobarse

CONVENTIONAL COMMENTS EN CODE REVIEWS

Al hacer code reviews, es útil seguir una convención para los comentarios, facilitando la comunicación y la acción.

- **Conventional Comments** propone prefijos como:

- **nitpick:** sugerencia menor
- **suggestion:** propuesta de mejora
- **question:** consulta o duda
- **praise:** reconocimiento positivo
- **issue:** problema a corregir

Ejemplo: > suggestion: Considera extraer esta lógica a una función aparte para mayor claridad.

Beneficios: - Claridad y contexto inmediato para quien recibe el feedback - Facilita la priorización y respuesta a los comentarios

Más info: conventionalcomments.org

USE CASES

Desarrollo de software

- Ampliamente utilizado para el desarrollo colaborativo de software en proyectos de cualquier tamaño

Proyectos de código abierto

- Muchos proyectos de código abierto utilizan GitHub como plataforma para alojar y gestionar sus repositorios

GITHUB'S ROLE IN TODAY'S IT WORLD

GitHub ha tenido un impacto significativo en el mundo IT:

- Ha transformado la forma en que se desarrolla y colabora en el software
- Ha fomentado la cultura del desarrollo de código abierto
- Ha contribuido a la disponibilidad de una gran cantidad de proyectos de código abierto
- Ha impulsado la transparencia y la accesibilidad en el desarrollo de software
- Se ha convertido en una referencia para la comunidad de desarrolladores

GITHUB FLOW

El flujo de trabajo de GitHub:

1. Todo lo que está en la rama **main** está listo para ser puesto en producción
2. Para trabajar en algo nuevo, crear una nueva rama a partir de main con un nombre descriptivo
3. El trabajo se irá integrando sobre esa rama en local y regularmente también en el servidor
4. Cuando se necesite ayuda o la rama esté lista, abrir una **pull request**
5. Alguien debe revisar y visar los cambios para fusionarlos con la rama main
6. Los cambios integrados se pueden poner en producción

DOCKER

INTRODUCTION

Docker es una plataforma de virtualización de contenedores que permite el empaquetado y despliegue de aplicaciones de manera eficiente.

- Fue lanzada en 2013
- Se ha convertido en una herramienta popular en el mundo del desarrollo de software y las operaciones de TI

VIRTUALIZATION

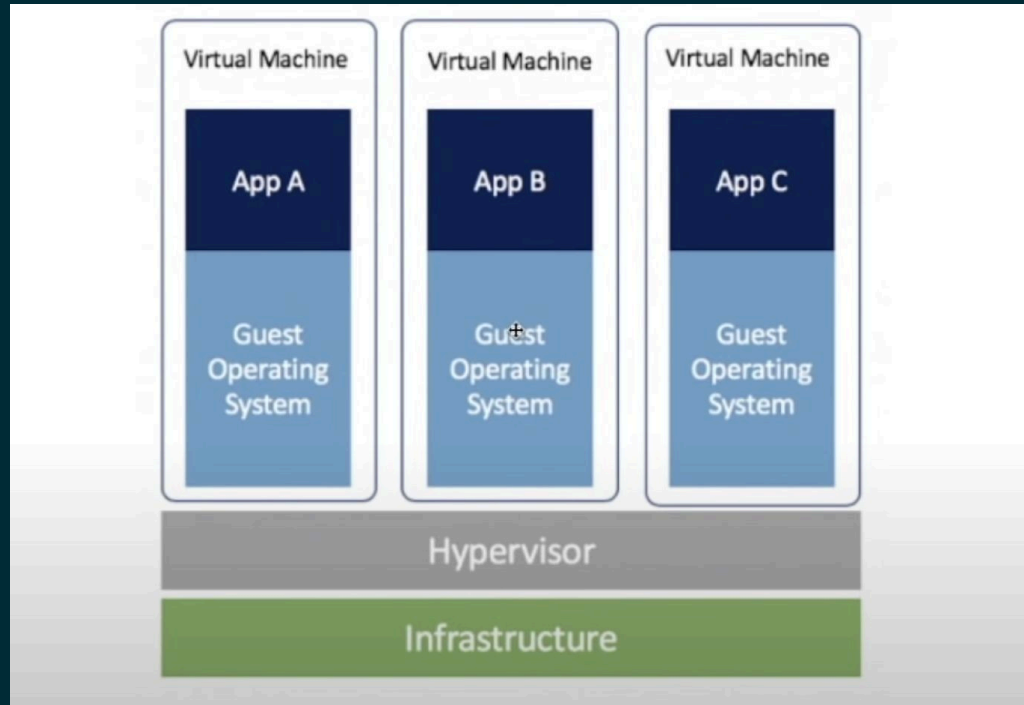
Virtualización tradicional:

- Emulación completa de hardware
- Múltiples sistemas operativos sobre un hipervisor
- Mayor consumo de recursos

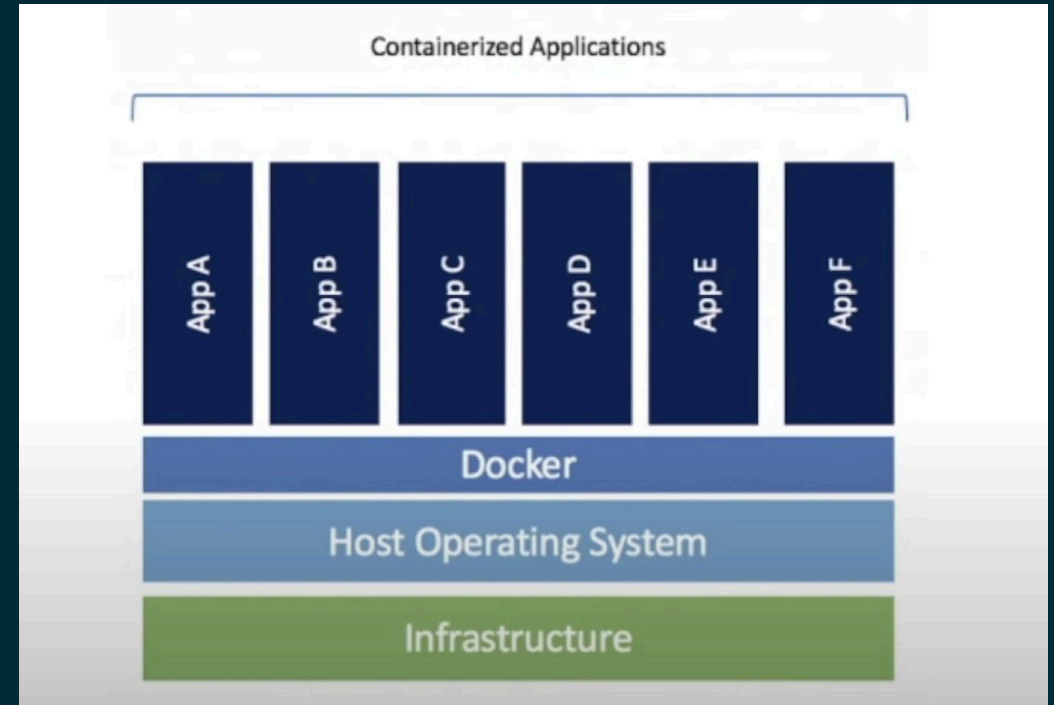
Contenedores:

- Virtualización a nivel de sistema operativo
- Comparten el kernel del host
- Más ligeros y eficientes

VIRTUALIZATION (CONTINUATION)



Virtual machine



Containerized Application

DOCKER BASICS

Conceptos base para empezar con Docker:

DOCKER ARCHITECTURE

Docker está compuesto por:

- **Docker Client:** interfaz CLI (**docker** commands)
- **Docker Daemon:** servicio de fondo que gestiona contenedores e imágenes
- **Docker Images:** plantillas versionadas e inmutables
- **Docker Containers:** instancias aisladas en ejecución
- **Docker Registry:** almacén central (Docker Hub, private registries)

Flujo típico:

1. Cliente envía comando al Daemon
2. Daemon verifica/descarga imagen del Registry
3. Daemon crea y ejecuta contenedor

DOCKER IMAGE LAYERS

Cada layer es una capa de cambios sobre la capa anterior.

Cómo Docker construye imágenes:

- `FROM nginx:alpine` → capa base
- `RUN apt-get install curl` → nueva capa
- `COPY . /app` → nueva capa
- `ENTRYPOINT ["nginx"]` → configuración

Ventajas:

- Reutilización de capas (reduce almacenamiento)
- Construcción caché (más rápido)
- Reproducibilidad garantizada

CONTAINER AS PROCESS

Un contenedor es essentially un proceso aislado con su propio ambiente.

- No es una máquina virtual: no arranca un SO completo
- Usa **namespaces** del kernel para aislar:
 - **pid**: aislamiento de procesos
 - **net**: stack de red privado
 - **ipc**: comunicación inter-procesos
 - **uts**: hostname y domainname
 - **user**: mapeo de UID/GID
 - **mnt**: punto de montaje aislado

Resultado: proceso “aislado” pero ágil y liviano.

ACCESSING RUNNING CONTAINERS

Para interactuar con un contenedor en ejecución:

```
1 # ejecutar comando dentro del contenedor
2 docker exec -it <container-id> /bin/bash
3
4 # ver logs del contenedor
5 docker logs <container-id>
6
7 # inspeccionar detalles
8 docker inspect <container-id>
9
10 # copiar archivos
11 docker cp <container-id>:/app/file.txt .
```

DOCKER DATA VOLUMES

Un data volume permite persistencia o compartir datos entre host y contenedor.

Tipos:

- **Bind mount:** mapea directorio host a contenedor (**-v /host/path:/container/path**)
- **Named volume:** volumen gestionado por Docker
- **Anonymous volume:** temporal, se borra con el contenedor

Cuándo usar:

- Persistir base de datos (no en **/tmp** del contenedor)
- Compartir código entre host e contenedor en desarrollo
- Separar datos (stateful) de la aplicación (stateless)

GOLDEN IMAGE & AMBIENTES REPRODUCIBLES

Una **golden image** es una imagen base confiable, versionada e inmutable.

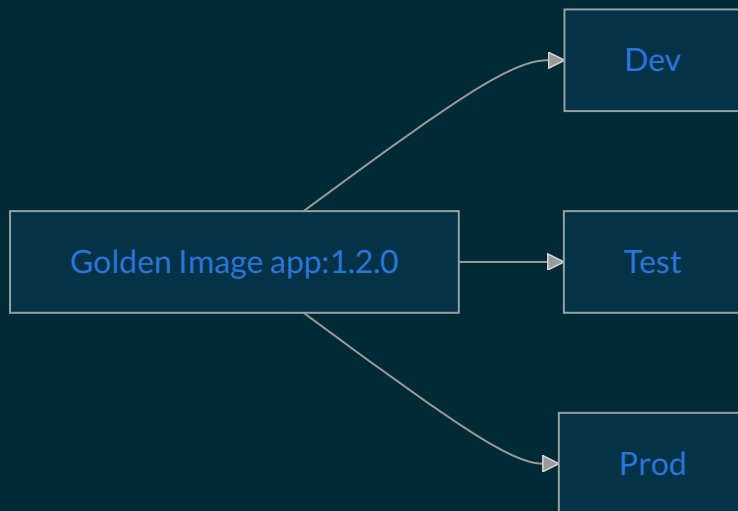
Por qué importa:

- Reduce diferencias entre desarrollo, testing y producción
- Mejora seguridad y consistencia (mismo stack en todos los entornos)
- Facilita rollback usando tags/versiones conocidas

Prácticas recomendadas:

- Versionar imágenes (**app:1.2.0**) y evitar usar solo **latest**
- Promover la misma imagen entre ambientes

GOLDEN IMAGE & AMBIENTES REPRODUCIBLES (CONTINUACIÓN)



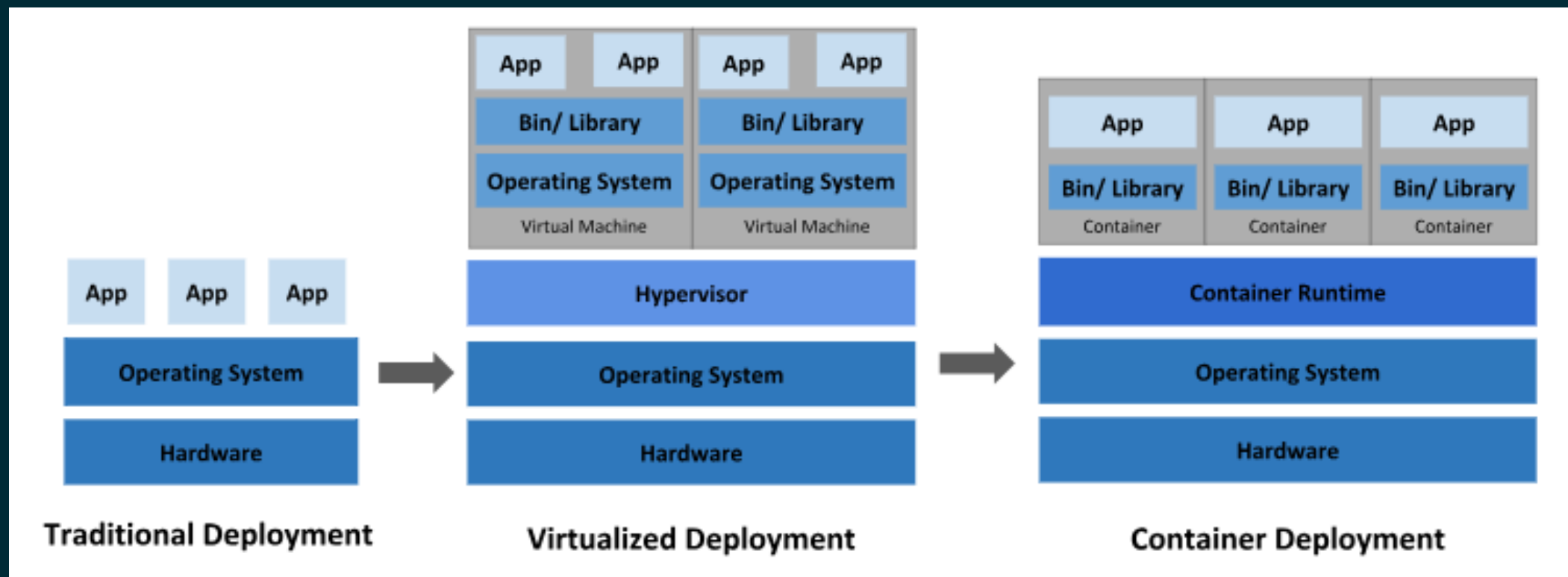
DOCKER EXAMPLE (DOCKERFILE + RUN)

Ejemplo mínimo para empaquetar una app web estática:

```
1 FROM nginx:alpine
2 COPY . /usr/share/nginx/html
3 EXPOSE 80
```

```
1 docker build -t demo-web:1.0 .
2 docker run --rm -p 8080:80 demo-web:1.0
```

TRADITIONAL DEPLOYMENT



DOCKER VS VIRTUAL MACHINES

Aspecto	Contenedores Docker	Virtual Machines
Qué es	Emulación de una instancia de espacio de usuario	Emulación de una máquina física completa
Virtualización	Abstrae los detalles del SO del código	Abstrae los detalles del hardware
Objetivo	Mejorar administración del entorno y uniformidad	Abstraer hardware y aumentar su utilización
Administrado por	Docker Engine	Hipervisor
Arquitectura	Comparte recursos con el kernel del host	Ejecuta su propio kernel y SO
Recursos	Bajo demanda	Cantidad fija establecida en la configuración
Kernel	Comparte kernel del host (namespaces + cgroups)	Kernel privado por VM

DOCKER ADVANTAGES OVER VMS

Ventajas de contenedores over tradición VMs:

- **Rapidez:** arranque en milisegundos (no segundos)
- **Overhead bajo:** megabytes (no gigabytes de RAM)
- **Densidad:** cientos/miles por máquina física
- **Portabilidad:** misma imagen garantiza mismo behavior everywhere
- **CI/CD ágil:** más fácil de construir, testear y distribuir
- **Escalabilidad horizontal:** spin up/down en segundos

Desventaja: - Todos en el mismo host comparten el kernel Linux - No tan aislados como VMs si hay vulnerabilidad en kernel

CANONICAL DEFINITION OF CONTAINER

Un contenedor Docker es:

- Un **proceso** ejecutado con aislamiento de namespaces y límites de recursos (cgroups)
- Que corre la aplicación + runtime + dependencias en un entorno reproducible
- Que arranca en milisegundos (no segundos)
- Que puede ser promovido sin cambios entre ambientes (dev → staging → prod)

NO es: - Una máquina virtual completa - Un nuevo SO - Un proceso sin restricciones

KEY DOCKER FEATURES

- **Contenedores:** Ligeros y aislados, empaquetan aplicaciones y sus dependencias
- **Portabilidad:** Se pueden ejecutar en diferentes entornos sin problemas
- **Eficiencia:** Virtualización a nivel de SO, más liviana que VMs tradicionales
- **Escalabilidad:** Facilita la escalabilidad horizontal mediante replicación
- **Gestión de imágenes:** Eficiente creación, distribución y actualización de aplicaciones

USE CASES

Despliegue de aplicaciones

- Encapsula aplicaciones con todas sus dependencias
- Facilita la replicación y la portabilidad

Entornos de desarrollo

- Crea entornos consistentes y reproducibles
- Elimina inconsistencias entre configuraciones

Pruebas y QA

- Proporciona entornos aislados y controlados
- Asegura la calidad del software

USE CASES (CONTINUACIÓN)

Microservicios

- Ideal para arquitecturas de microservicios
- Cada servicio se ejecuta en su propio contenedor independiente

Actualizaciones y escalabilidad

- Permite actualizaciones y escalabilidad más rápida
- Administración sencilla de contenedores

DOCKER IN TODAY'S IT WORLD

Docker ha tenido un gran impacto en el mundo IT:

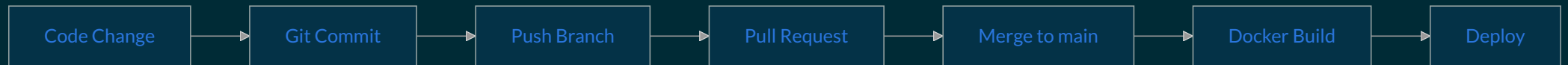
- Ha facilitado la adopción de la infraestructura como código y las prácticas de DevOps
- Ha mejorado la eficiencia y la agilidad en el desarrollo y despliegue de aplicaciones
- Ha fomentado la estandarización y la portabilidad en el mundo del software
- Ha contribuido a la adopción de arquitecturas de microservicios
- Se ha convertido en una herramienta fundamental en el desarrollo de aplicaciones modernas

DEVOPS WORKFLOW (END-TO-END)

Cómo se conectan Git, GitHub y Docker en un flujo real:

1. Desarrollar cambios en local
2. Versionar con Git (**add**, **commit**)
3. Publicar rama y abrir Pull Request en GitHub
4. Construir imagen Docker de la aplicación
5. Desplegar la misma imagen en diferentes entornos

DEVOPS FLOW DIAGRAM



PRÓXIMA CLASE

CI/CD

- Continuous Integration
- Continuous Delivery/Deployment